Web final project report

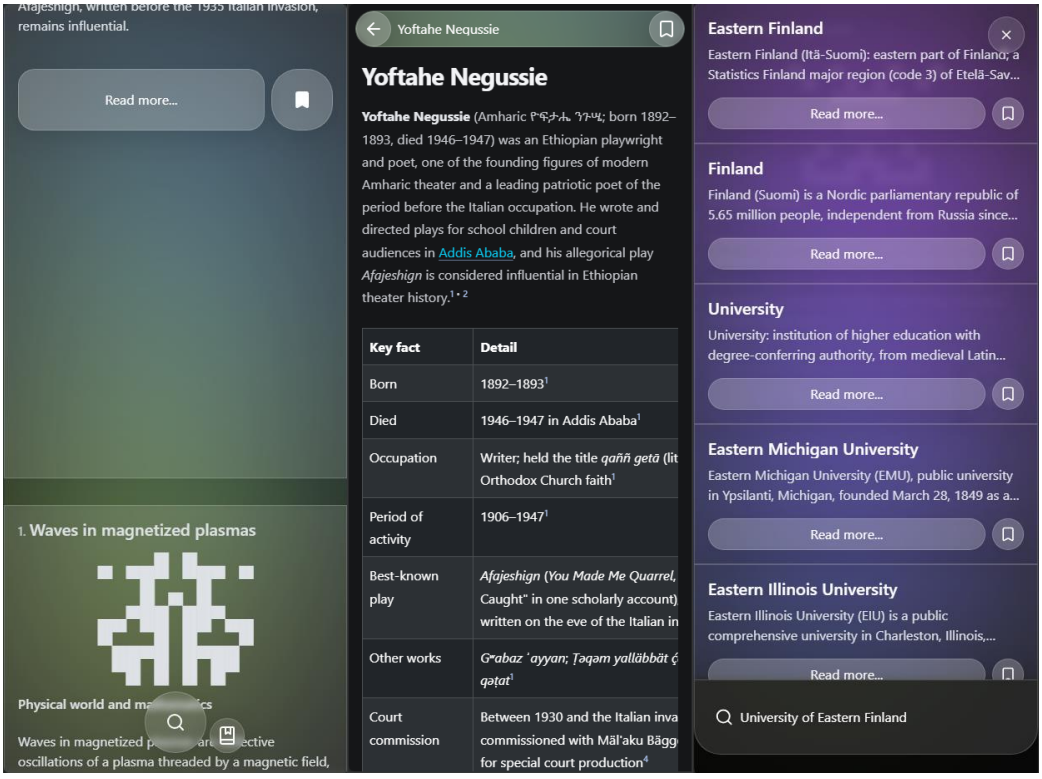
Course Web Programming I, Antonín Kadrmas

Project description

The application lets users discover new articles about various topics rapidly. By using infinite scroll feed with similar interaction to the short for content on social networks like Instagram, TikTok, or YouTube. The app was designed and developed as a mobile first app, so it is mainly for mobile browsers (works also in desktop). And targeted user is any user who likes to discover new information and broaden his/her knowledge even in topics that he/she wouldn't normally come across.

I am using edgepedia.org API from where I access a wide range of articles. Form Tanstack I markdown (markdown parsing), router (routing), virtual (virtual scroll), query (api management). Zustand (state management), embla (for feed), bootstrap (for styles), react bootstrap (components), lucide (icons), gradients, gh-pages (deployment), eslint (linting). There is a link to my GitHub and there are the GitHub pages.

Screenshots of the app



The fisrt screen shows the feed view while scrolling to the next article.

The second screen shows the aticle view where is the content of specific article.

The third screen shows the search view where is articles that were returned for te search text in bottom.

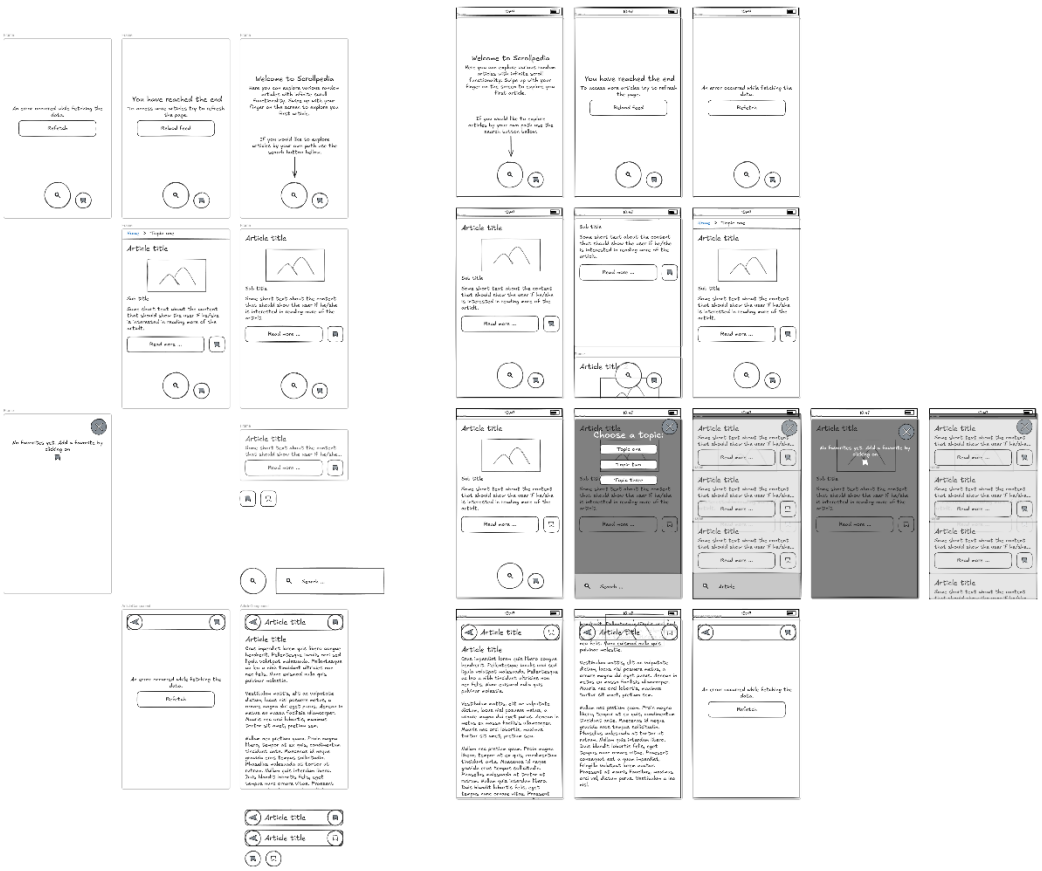
AI usage statement

Used AI agents sometimes just chatbots. Firstly, while integrated the tanstack markdown library I used it to help me style the citation numbers inside the markdown. I also used it to create styles for the markdown based on the reference that I provided. I used it for mobile responsivity to help me define the sizes of containers correctly based on the safe areas through whole application. While refactoring to point out where the error is so I could fix it by myself. I did not use AI while creating app logic, components, basic styling, api logic, state management, refactoring (until some error occurred that I stacked on for longer period), debugging, because I wanted to do it by myself.

Reflection

One of the most challenging parts of the project was choosing the project topic of the final and finding some interesting API. So, I solved it by going into this GitHub repo that contains many APIs that can be accessed categorized by different topics. Also deciding the appropriate scope of the project so I am not creating something unnecessary large (I thought that I would also add sign in and some other features) so the web application is the MVP that works great and reliably instead of some huge app that would not be tested enough and could fall because of the time press.

For the structure of components mostly helped the design that I created in the excalidraw.



Some other components have been created while refactoring bigger files into smaller ones.

I knew from begging that I would like to use tanstack react query for the api state management, so when other needs like the markdown parsing, virtual scroll, or routing management I also used the tanstack library (to broaden my skills). For the state management I sued the zustand as I have some experience with it and I like it. For the ascii avatar component I decided during the development as I did not have any image to display from the design and without it the text was too close. I did work with embla carousel in past and it looked like a good approach for the feed implementation.

I would add the account creation and save the favorite into some backend. I find out there are also books in the api so maybe I would somehow incorporate also that. I would do more testing. In some cases, be more specific about the states the application is in (now it is quite general e.g. the errors are considered in app as network errors). I would go maybe for some cleaner more thoughtful colors and design decisions. Not sure how the feed would work with too many elements so maybe I would incorporate the virtual scroll also in there. Maybe I would also work with the husky and commitlint to ensure the compliance with conventional commits, app linting before a new commit, to pushing just buildable application, and maybe deploy on push?